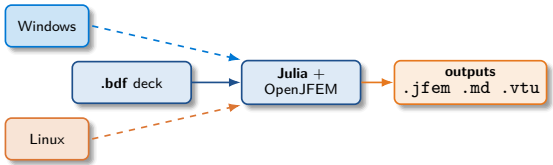
Running the JFEM Solver

User reference guide for Windows and Linux
Launch options, output formats, batch runs, sysimage

JFEM

Universidad de Sevilla

Updated October 15, 2026



Outline

1. What JFEM does — 60-second overview
2. The three-stage execution model
3. Prerequisites and first-time setup
4. Anatomy of a JFEM launch command
5. Launching on **Windows** (PowerShell)
6. Launching on **Linux** (Bash helpers)
7. Output format selection
8. Enabling HDF5 / VTK / JSON
9. Run-time environment flags
9. Precompilation `deploy_fast.jl`
10. Sysimage generation with PackageCompiler
11. Running with a sysimage
12. Single-case launcher `run_bdf.jl`
13. Interactive web app `panel_app`
14. Batch runs via JSON manifests
15. Persistent JSONL worker (Python loops)
16. Recommended recipes by user type
17. Troubleshooting and quick reference

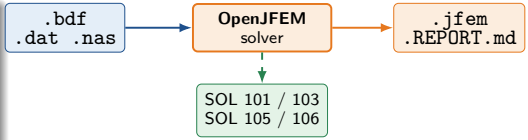
What JFEM does — 60-second overview

Solver scope

JFEM (`OpenJFEM`) is a Julia finite-element solver with a **Reference (BDF)** front end. Supported solution sequences:

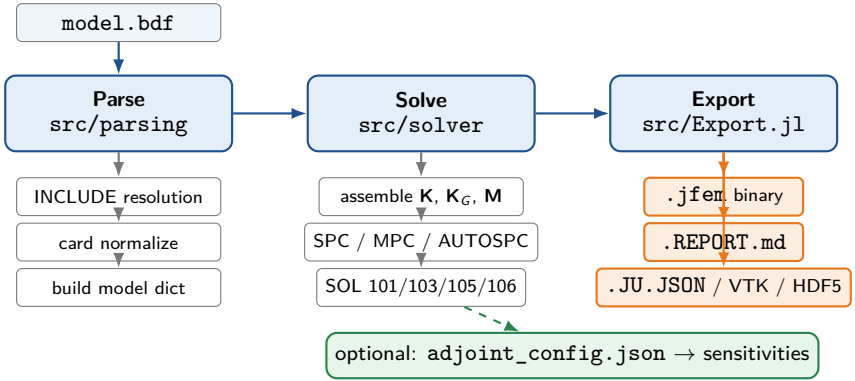
- `SOL 101` — linear static
- `SOL 103` — normal modes
- `SOL 105` — linear buckling
- `SOL 106` — nonlinear static (experimental)

It reads `.bdf`, `.dat`, or `.nas` decks and writes text/binary results plus a Markdown report.



Designed for **batch buckling** and **Python-driven sizing loops**, with first-class precompilation and sysimage support.

Three-stage execution model: **parse** → **solve** → **export**



Everything you launch ultimately calls `OpenJFEM.main(deck;...)` which executes these three stages in order.

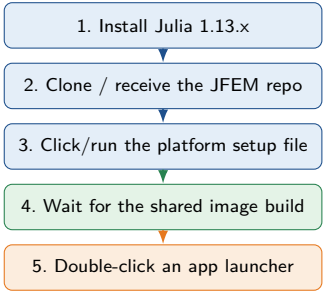
Prerequisites and first-time setup

What you need

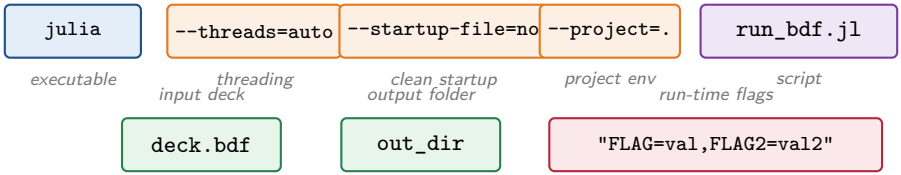
- **Julia 1.13.x** on PATH (or set `JFEM_JULIA=/path/to/julia`)
- **Git** (only for cloning / pulling JFEM)
- A **bulk-data deck**: `.bdf`, `.dat`, or `.nas`
- A working directory for outputs

Disk / memory rule of thumb

- Setup installs packages and compiles one shared image
- First build can take several minutes or longer; progress is logged
- Solve: dominated by SOL type and mesh size



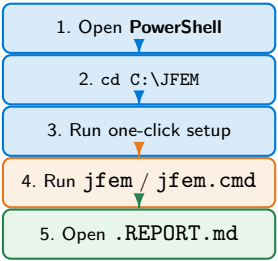
Anatomy of a JFEM launch command



Read left to right

Blue = Julia executable. **Orange** = Julia flags that should not change. **Purple** = the JFEM tool you call. **Green** = your data (input deck, output folder). **Red** = a comma-separated string of `JFEM_*` environment flags — keep the quotes.

Launching on Windows — PowerShell



First-time setup (once after install/update):

```
cd C:\JFEM
.\setup.cmd
```

Single buckling case:

```
.\jfem C:\models\panel.bdf D:\runs\panel
```

The PowerShell backtick ‘ continues the command on the next line. On a single line, omit it.

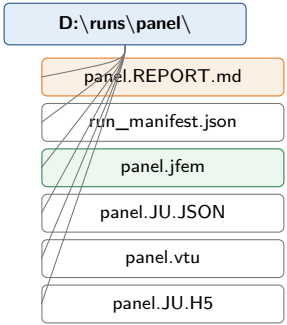
Windows: where outputs go

Output folder layout

After a successful run the output directory contains:

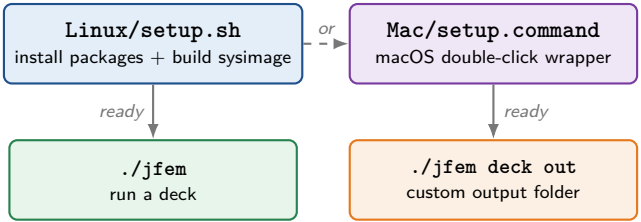
```
D:\runs\panel\
  run_manifest.json # what was launched
  panel.REPORT.md # human-readable report
  panel.jfem # binary (if enabled)
  panel.JU.JSON # results JSON (if enabled)
  panel.vtu # VTK (if enabled)
  panel.JU.H5 # HDF5 (if enabled)
```

The Markdown report is the main file to open first: model counts, active flags, solver timings, and (for SOL 105) the buckling load factors.



only the toggles you enable are written

Launching on Linux/macOS — setup and run



From the repository root:

`bash setup.sh`

Then `bash jfem /path/to/deck.bdf`

Public launcher location

`__Start here!__/Linux/setup.sh`
`__Start here!__/Mac/setup.command`

The repository-level `jfem` launcher automatically uses `sysimage/OpenJFEM_sysimage.so` or `.dylib` when its provenance receipt matches the checkout and runtime.

Linux/macOS: typical usage

First-time setup (one-time):

```
bash setup.sh
```

Add a representative deck to the automatic image build:

```
bash setup.sh --sysimage --deck /projects/.../MODELS/panel105.bdf
```

Run a deck:

```
./jfem /projects/.../MODELS/panel105.bdf /runs/panel105 # custom output
```

Setup installs one shared package environment for all applications and builds their shared sysimage automatically.

Advanced users can opt out with `--no-sysimage`.

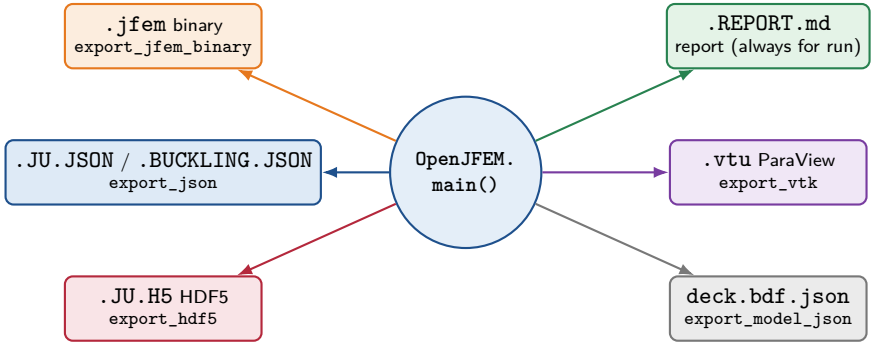
Side-by-side: Windows vs. Linux commands

Step	Windows PowerShell	Linux/macOS Bash
First setup	`.\setup.cmd`	`bash setup.sh`
Setup with representative deck	`.\setup.cmd --deck C:\m.bdf`	`bash setup.sh --deck /m.bdf`
Build sysimage	`included in setup launcher`	`included in setup launcher`
Single run	`.\jfem.cmd deck.bdf out\dir`	`./jfem /m.bdf [/out/dir]`
Single run + sysimage	`.\jfem.cmd deck.bdf out\dir`	`./jfem /m.bdf [/out/dir]`
Batch run	Use `run_batch_manifest.jl`; see the complete quoted commands on the "Running a batch manifest" slide.	

One mental model

The public launchers call the same Julia helper scripts underneath. Users normally run one setup file once, then call `jfem` or `jfem.cmd` for day-to-day solves.

Output format selection — the six toggles



Default in the root `jfem` / `jfem.cmd` launchers

The viewer-ready set is `.jfem` binary, Markdown report and results JSON. Use an output-format string (such as `-jrsvh`) or a JSON manifest to choose outputs.

Output formats — when to use which

Format	Extension	When to use
JFEM binary	`.jfem`	Full results in compact form. Reload with the `APPS/JFEM_results_viewer/postv11.html` browser viewer. Set `JFEM_EXPORT_BINARY=false` to skip for max throughput.
Markdown report	`.REPORT.md`	Always start here. Lists timing, model counts, active flags, and (SOL 105) buckling load factors.

Results JSON	`.JU.JSON`
	`.BUCKLING.JSON`
	`.NONLINEAR.JSON`
		Machine-readable results for Python, MATLAB, or downstream sizing tools. Name depends on the SOL.

ParaView VTK	`.vtu`	Visual checks: mode shapes, displacements, stress fields. Skip for headless production.
HDF5	`.JU.H5`	Large structured results with attributes; ideal for archival or HPC pipelines.
Model JSON	`deck.bdf.json`	A snapshot of the parsed/built model before solving — useful for debugging the BDF interpretation.
Card inventory	`.cards.json`	Audit of every Reference card recognized in the deck; helpful when migrating decks from other solvers.

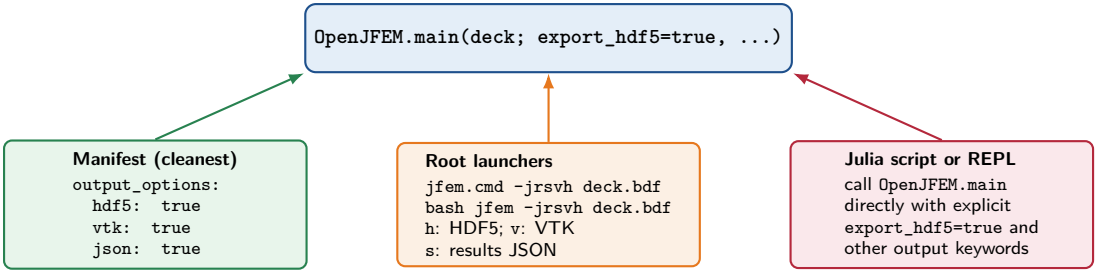
SOL105 element strain energy and density

Use Case Control output requests before `BEGIN BULK`. A global request applies to the static preload and mode subcases; a local request overrides it.

```
SET 42 = 101, 102, 103
ESE(PLOT) = ALL
SUBCASE 1
  LOAD = 1
  SPC = 1
SUBCASE 2
  STATSUB = 1
  METHOD = 1
  ESE(PLOT) = 42
```

- `ALL`: all supported elements; `42`: the element SET above; `NONE`: suppress output for that scope.
- ESE requests both $U_e = \frac{1}{2} u_e^T K_e u_e$ and density U_e / V_e. Undefined volume means unavailable density, not zero.
- `PRINT/PLOT/PUNCH` qualifiers are accepted; JFEM writes these fields in `.BUCKLING.JSON`. Modal values depend on eigenvector normalization.
- `PARAM,JFBKCOMP,1` overrides energy export with compact root and peak-node/property metadata. It leaves the eigenproblem unchanged.

Enabling HDF5, VTK, and results JSON



The underlying knob

All three paths ultimately call `OpenJFEM.main(deck; ...)` with the boolean keywords `export_hdf5`, `export_vtk`, `export_json`, `export_jfem_binary`, `export_report`, `export_model_json`, `export_card_inventory`. All default to `false` except `export_jfem_binary` (and `export_report` in the helpers).

Enable HDF5 — manifest or launcher

1. JSON batch manifest (preferred for repeatable runs):

```
"defaults": {
  "output_options": {
    "binary": false,
    "json": true,
    "hdf5": true,
    "vtk": true,
    "report": true
  }
}
```

Aliases accepted by the manifest reader: `hdf5`, `h5`, `export_hdf5` / `vtk`, `export_vtk` / `json`, `export_json`, `results_json`.

Caveat: if `eigenvalues_only: true` is set for SOL 105, HDF5 and VTK are forced off.

2. Root launcher format string (`j`: binary, `r`: report, `s`: JSON, `v`: VTK, `h`: HDF5):

```
.\jfem.cmd -jrsvh C:\models\panel.bdf D:\runs\panel
bash jfem -jrsvh /models/panel.bdf /runs/panel
```

The first command is for Windows PowerShell; the second is for Bash.

Enable HDF5 — direct Julia API

3. Save a Julia script, for example `analysis.jl`:

```
using OpenJFEM
OpenJFEM.main("C:/models/panel.bdf";
    output_dir="D:/runs/panel",
    export_json=true,
    export_vtk=true,
    export_hdf5=true,
    export_jfem_binary=true,
    export_report=true,
)
```

Run it from the repository root (Windows, Linux or macOS):

```
julia --threads=auto --startup-file=no --project=. analysis.jl
```

`utils/julia_tools/run_bdf.jl` is the explicit helper behind the public launcher. For richer output-control workflows, prefer a JSON manifest or call `OpenJFEM.main` directly.

Run-time environment flags (JFEM_*)

Flag	Default	Effect
JFEM_EXPORT_BINARY	true	false skips .jfem for max solver throughput.
JFEM_MATRIX_ASYMMETRY_CHECK	true	false skips an expensive buckling diagnostic.
JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES	true	false drops the duplicate mode-shape array from the returned dict.
JFEM_SOL105_EIGENVALUES_ONLY	false	true keeps only the eigenvalues (no mode vectors).
JFEM_SUPPRESS_THREAD_HINT	0	1 suppresses the single-thread warning in logs.
JFEM_JULIA (shell var)	—	Path to a non-default Julia executable.

Recommended *fast* string (single-case CLI)

```
"JFEM_EXPORT_BINARY=false,JFEM_MATRIX_ASYMMETRY_CHECK=false,JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES=false,
     JFEM_SUPPRESS_THREAD_HINT=1"
```

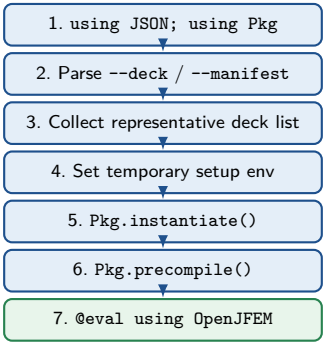
Precompilation — why and how

Why precompile

Julia compiles methods the first time it sees each new combination of types and code paths. Without precompilation, the *first* JFEM run pays minutes of compile time. `deploy_fast.jl` warms those paths upfront so later runs are fast.

What it touches

- `Pkg.instantiate()` — restore dependencies
- `Pkg.precompile()` — compile all packages
- Loads `using OpenJFEM` and runs a representative workload



Setup launcher options

Option	Meaning
`--sysimage`	Build the shared image (already the default).
`--deck <path>`	Add a representative deck; used by the automatic image build.
`--manifest <path>`	Use a JSON cases workload; used by the automatic image build.
`--flags FLAG=val,...`	Set workload flags; used by the automatic image build.
`--sysimage-threads=N`	Select build-child threads; 2 is the default to limit compiler memory.

Image workload sources (all setup launchers build the shared image):

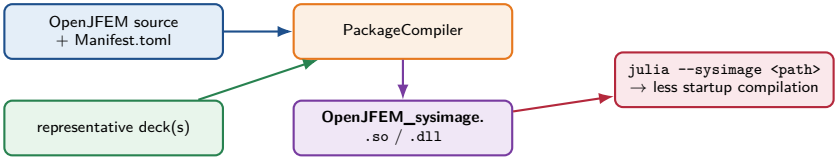
```
# 1. Default (bundled utils/examples/precompile/*.bdf)
.\setup.cmd

# 2. Representative deck
.\setup.cmd --deck C:\models\panel105.bdf

# 3. Manifest
.\setup.cmd --manifest C:\models\cases.json
```

Rule of thumb: use a deck that *resembles* the cases you will run in production.

Sysimage — ahead-of-time compilation



What a sysimage is

A native shared library storing the solver, JFEM-Boxer and their common packages and selected compiled methods. It can reduce startup latency; new code paths may still compile. Verify the receipt before direct use.

Build (Windows):

```
.\setup.cmd `
     --deck C:\models\panel105.bdf
```

Build (Linux/macOS):

```
bash setup.sh --deck /projects/.../MODELS/panel105.bdf
```

Sysimage: when to (re)build it

Rebuild required

- OpenJFEM source code changed
- `Project.toml` / `Manifest.toml` changed
- Julia runtime / package update or checkout move
- Different OS / CPU architecture
- Representative build deck changed or removed

Reuse if receipt still matches

- Running new decks of the same SOL
- Output toggles changed
- `JFEM_*` env flags changed
- Different mesh sizes
- Different material data

Default sysimage path

Linux/macOS launchers: `<repo>/sysimage/OpenJFEM_sysimage.so` / `.dylib`
Windows: typically `sysimage\OpenJFEM_sysimage.dll`

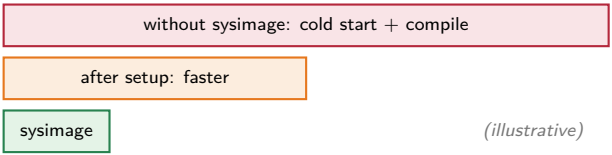
Running with a sysimage

Windows (single case):

```
.\jfem C:\models\panel.bdf D:\runs\panel
```

Linux/macOS:

```
./jfem /projects/.../MODELS/panel.bdf /runs/panel
```



Single-case launcher `jfem` / `jfem.cmd`

Signature:

```
jfem <input.bdf> [output_dir]
```

Behavior:

- Creates `output_dir` if missing.
- Writes a `run_manifest.json` capturing the launch parameters.
- Calls the public Julia runner under `utils/julia_tools/`.
- Automatically uses `sysimage/OpenJFEM_sysimage.*` when its runtime and source receipt matches.

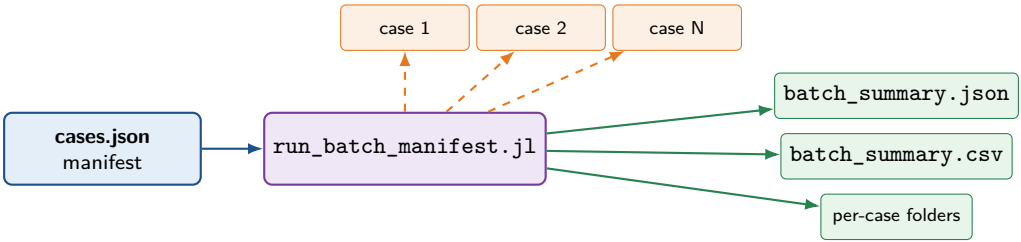
Example (Windows):

```
.\jfem C:\models\panel_001.bdf D:\runs\panel_001
```

Example (Linux/macOS):

```
./jfem /projects/.../panel_001.bdf /runs/panel_001
./jfem /projects/.../panel_001.bdf /runs/panel_001
```

Batch runs — JSON manifest is the preferred interface



Why a manifest beats shell loops

Every input deck, output folder, flag, and output toggle is explicit. The same JSON file feeds command-line, Python loops, and job schedulers, and avoids quoting bugs.

Minimal SOL 105 manifest

```
{
  "batch_id": "batch_001",
  "output_root": "D:/jfem_runs/batch_001",
  "defaults": {
    "flags": {
      "JFEM_EXPORT_BINARY": "false",
      "JFEM_MATRIX_ASYMMETRY_CHECK": "false",
      "JFEM_SOL105_STORE_PUBLIC_MODE_SHAPES": "false",
      "JFEM_SUPPRESS_THREAD_HINT": "1"
    },
    "output_options": {
      "binary": false,
      "json": true,
      "report": true
    },
    "gc_between": true,
    "stop_on_error": false
  },
  "cases": [
    { "case_id": "panel_001", "input": "C:/models/panel_001.bdf",
      "output_dir": "D:/jfem_runs/batch_001/panel_001" },
    { "case_id": "panel_002", "input": "C:/models/panel_002.bdf",
      "output_dir": "D:/jfem_runs/batch_001/panel_002" }
  ]
}
```

/ works as path separator on Windows too — avoids escaping backslashes.

Manifest reference — fields and effects

Field	Meaning
`batch_id`	Readable name; appears in summaries.
`output_root` *(required)*	Batch-level output folder.
`defaults.flags`	Map of `JFEM_*` env values applied to every case.
`defaults.output_options.binary`	Toggle `.jfem` export per case.
`...output_options.json`	Toggle results JSON.
`...output_options.vtk`	Toggle ParaView `.vtu`.
`...output_options.hdf5`	Toggle HDF5.
`...output_options.report`	Markdown report (default `true`).
`...output_options.eigenvalues_only`	SOL 105 — discard eigenvectors.
`...output_options.eigenvectors`	SOL 105 — keep mode shapes.
`defaults.gc_between`	Force GC between cases (memory hygiene).
`defaults.stop_on_error`	Abort batch on first failure.
`cases[].case_id`	Stable name in summaries; can repeat slug with suffix.
`cases[].input` *(required)*	`.bdf`/`.dat`/`.nas` path.
`cases[].output_dir`	Custom output folder (otherwise `output_root/slug`).
`cases[].flags` / `cases[].output_options`	Per-case overrides.

Running a batch manifest

Windows:

```
julia --threads=auto --startup-file=no --project=. ‘
     ’.\utils\julia_tools\run_batch_manifest.jl’ C:\models\cases.json --quiet
```

Linux:

```
julia --threads=auto --startup-file=no --project=. \
     ’./utils/julia_tools/run_batch_manifest.jl’ /home/user/models/cases.json --quiet
```

Useful flags on the batch runner:

- `--quiet` keeps per-case solver output out of stdout; each case's chatter goes to `<case>/jfem_case_stdout.log`.
- `--stop-on-error` aborts the batch on the first failure (overrides manifest setting).

Outputs written:

```
<output_root>/batch_summary.csv
<output_root>/batch_summary.json
<output_root>/<case_id>/run_manifest.json
<output_root>/<case_id>/<input_stem>.REPORT.md
<output_root>/<case_id>/<input_stem>.JU.JSON # if json=true
<output_root>/<case_id>/jfem_case_stdout.log
```

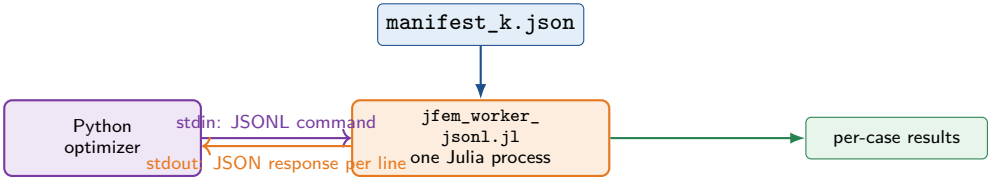
Legacy text/CSV batch (`run_bdf_batch.jl`)

Argument / option	Meaning
`<input>` (positional)	One `.bdf`, a directory of decks, or a text/CSV list (one path per line).
`<output_root>` (positional)	Top-level output folder.
`FLAG=val,...` (positional)	Flag string, same shape as single-case runner.
`--recursive`	Walk subdirectories.
`--ext=.bdf,.dat`	Override accepted deck extensions.
`--stop-on-error`	Stop on the first failed case.
`--dry-run`	List cases without solving.
`--no-gc-between`	Skip the explicit GC step between cases.

When to use which batch tool

New automation: use the JSON manifest (`run_batch_manifest.jl`).
Quick ad-hoc sweep over a folder of decks: `run_bdf_batch.jl` is fine.
Python sizing loops: prefer the JSONL worker (next slide).

Persistent JSONL worker for Python optimization loops



Why keep Julia open

Restarting Julia per design iteration pays minutes of compile/load each time. The worker stays loaded, accepts JSONL commands, and runs manifests in a single process — the recommended architecture for hot Python sizing loops.

JSONL commands accepted: `status` / `ping`, `gc`, `run_batch` (with `manifest`), `quit`.

Starting the JSONL worker

Windows:

```
julia --threads=auto --startup-file=no --project=. ‘
     ’.\utils\julia_tools\jfem_worker_jsonl.jl’
```

Linux:

```
julia --threads=auto --startup-file=no --project=. \
     ’./utils/julia_tools/jfem_worker_jsonl.jl’
```

Example exchange on stdin/stdout:

```
{"command":"status"}
{"command":"run_batch","manifest":"C:/models/iter_007.json"}
{"command":"gc"}
{"command":"quit"}
```

stdout discipline: stdout is reserved for JSON responses (one per line). Solver chatter is written to `jfem_case_stdout.log` in each case's output folder; human-readable diagnostics go to stderr.

Recommended recipes by user type

Engineer / quick run

1. `setup launcher` (once)
2. `./jfem deck.bdf`
3. Open `deck.REPORT.md`

Shared image selected automatically.

Production batch

1. `setup launcher --deck rep.bdf`
2. Write `cases.json`
3. `run_batch_manifest.jl`

Manifest = single source of truth.

Python sizing loop

1. `setup launcher --deck rep.bdf`
2. Start `jfem_worker_jsonl.jl`
3. Stream `run_batch` commands

One Julia process for thousands of iters.

Debugging a deck

1. `./jfem deck.bdf`
2. Inspect `deck.bdf.json`
3. Inspect `deck.REPORT.md`

Use `export_model_json=true`.

Visual review

1. Run with `export_vtk=true`
2. Open `.vtu` in ParaView
3. Or open `.jfem` in the results viewer

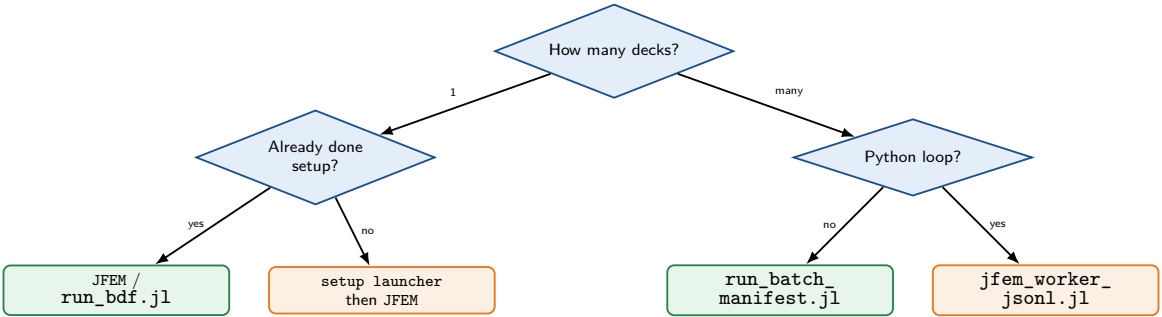
Open the root `Results-viewer.cmd` launcher.

Sensitivities

1. Place `adjoint_config.json` next to the deck
2. Run normally
3. Read `.ADJOINT.JSON`

Triggered after the forward solve.

Decision tree: which launcher should I use?



One shared image for all launchers

Setup builds one image for the solver and apps from the root `Project.toml` and `Manifest.toml`. Launchers verify its receipt automatically. A long-lived JSONL worker also reuses its loaded solver between requests.

Common pitfalls and fixes

Symptom	Likely cause / fix
`Error: JFEM sysimage not found`	The image path does not exist. Rerun setup from your OS folder under `__Start here!__/`.
First run takes minutes	Julia is compiling. Run setup after install. New code paths may still compile; timing reports separate compilation from solving.
`'\' is not recognized` on Windows	PowerShell line-continuation is backtick ‘, not backslash. Or put the command on one line.
`Error: ... does not look like a Julia project`	`--project=.` was run outside the JFEM repo. `cd` into the repo first, or use `--project=C:\JFEM`.
Solver runs but no `.jfem` appears	`JFEM_EXPORT_BINARY=false` or `binary:false` in the manifest. Flip the toggle.
Batch run silent in console	`--quiet` routes per-case chatter to `jfem_case_stdout.log`.
Sensitivities not produced	`adjoint_config.json` must sit next to the input deck, not in the output folder.
Wrong Julia picked up	Set `JFEM_JULIA` to the Julia executable before calling launchers.
Slow buckling diagnostic	Set `JFEM_MATRIX_ASYMMETRY_CHECK=false`.
Memory growth in long batches	Keep `defaults.gc_between=true`; the worker also accepts a `{"command":"gc"}`.

Quick reference card

Task	Command (short form)
First-time setup	`.\setup.cmd` or `bash setup.sh`
Custom image workload	optionally add `--deck rep.bdf`
Single run	`.\jfem deck.bdf out_dir` or `./jfem deck.bdf out_dir`
Single run + sysimage	same command; standard image loads only when its receipt matches
Batch manifest	`...\utils\julia_tools\run_batch_manifest.jl cases.json`
Text/dir batch	`...\utils\julia_tools\run_bdf_batch.jl <input> <out_root>`
Python loop	`...\utils\julia_tools\jfem_worker_jsonl.jl` (one process, JSONL on stdin)

Three numbers to remember

1: one setup launcher after install. **3**: public launchers (`jfem.cmd`, `jfem`, and platform setup file). **6**: output toggles (`jfem_binary, report, json, vtk, hdf5, model_json`).

One-page cheat sheet

Windows — single SOL 105 case end-to-end:

```
cd C:\JFEM
.\setup.cmd --deck C:\models\panel105.bdf
.\jfem C:\models\panel.bdf D:\runs\panel
```

Linux — same scenario:

```
bash setup.sh --deck /projects/.../MODELS/panel105.bdf
./jfem /projects/.../MODELS/panel.bdf /runs/panel
```

Batch (cross-platform):

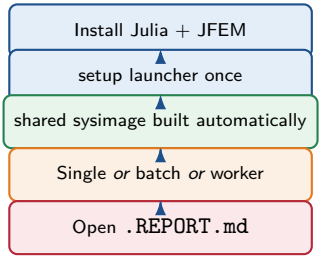
```
julia --threads=auto --startup-file=no --project=. './utils/julia_tools/run_batch_manifest.jl' ./cases.json --quiet
```

Python loop:

```
julia --threads=auto --startup-file=no --project=. './utils/julia_tools/jfem_worker_jsonl.jl' # then feed JSONL on
      stdin
```

Summary

- Click/run setup in the Windows, Linux or Mac folder under `__Start here!__/`.
- Windows, Linux, and macOS share the same Julia helper scripts underneath.
- Run the setup launcher **once** after install — it installs packages and builds the local sysimage.
- Rebuild the sysimage after Julia/package changes or major solver updates.
- Use the **JSON manifest** for batches; use the **JSONL worker** for hot Python loops.
- Toggle outputs deliberately: `.jfem` off for speed, on for archival.



One workflow, three operating systems, clear public launchers.

Happy solving!

Run one platform setup launcher once. Launch with confidence.

